

Contents

Contents	2
I. El Sindicato de la Resiliencia	3
II. Prólogo	4
III. Introducción	5
A. Ejercicio 0: El Guion de la Misión	6
B. Ejercicio 1: Limpiando la Escena del Crimen	7
C. Ejercicio 2: El Buscador de Mercenarios (Optional)	8
D. Ejercicio 3: El Almacén de Deadpool (I/O)	9
E. Ejercicio 4: Filtrado de Hechizos (Streams avanzados)	10
F. Ejercicio 5: Conversión de Datos (Map)	11
G. Ejercicio 6: Validación de Identidad (Optional + Filter)	12
H. Ejercicio 7: El Informe del Sindicato (JSON Reading)	13
G. Ejercicio 8: Exportación de Loot (JSON Writing)	15
A. Extra A: El Cuartel General (Collectors.groupingBy)	16
B. Extra B: El Botín Total (reduce)	17
C. Extra C: Sin Excusas (orElseThrow + flatMap)	18
Ejecución	20

I. El Sindicato de la Resiliencia

Has dominado la estructura básica y la resiliencia ante errores. Ahora, vamos a subir el nivel: aprenderás a manejar archivos en Java con la elegancia de un profesional. Este módulo te obliga a abandonar los bucles tradicionales para adoptar el paradigma funcional de Java moderno. Aprenderás que procesar archivos no es solo "leer y escribir", es **transmutar datos** mediante tuberías (Streams) que filtrarían incluso la personalidad de Wolverine. Prepárate, porque vamos a dejar el código tan limpio que hasta Coloso estará orgulloso.

Instrucciones Generales

- ✓ **Versión:** Java 17+ (OpenJDK).
- ✓ **Estructura:** Una clase por archivo (Puedes usar package).
- ✓ **Linter Obligatorio:** Checkstyle (Google Java Style). Errores de estilo = KO.

- ✓ **Comentarios:** Obligatorios, solo en inglés y siguiendo formato Javadoc.
- ✓ **Nomenclatura:** Clases en PascalCase, métodos y variables en camelCase.
- ✓ **IDE:** IntelliJ IDEA (recomendado), VS Code, Eclipse, etc.
- ✓ **Compilación:** Vía terminal: `javac Clase.java` y `java Clase`.

Entrega y Evaluación

- Entrega en repositorio Git.

II. Prólogo

Ser el socio de Deadpool no es tan emocionante como aparece en los cómics. Mientras él acapara las portadas, rompe la cuarta pared y convierte cada misión en un desastre espectacular, alguien tiene que encargarse del trabajo realmente importante: organizar toda la información que deja tras de sí.

Después de cada contrato aparecen decenas de archivos con listas de objetivos, informes de daños, facturas que nadie piensa pagar, registros de armas, coordenadas de extracción y notas escritas en servilletas manchadas de chimichangas. El problema es que Deadpool tiene muchas virtudes... pero el orden no es una de ellas.



Tu misión consiste en convertir ese caos en información útil. Tendrás que localizar archivos perdidos, eliminar registros duplicados, reparar datos incompletos y preparar informes antes de que lleguen a manos de clientes, héroes, villanos o, en el peor de los casos, Hacienda.

Por suerte, no lucharás solo. Contarás con las herramientas más modernas de Java para enfrentarte a cualquier montaña de datos. Los viejos bucles `for` fueron útiles en su momento, pero este trabajo exige algo más rápido y elegante. Los **Streams** recorrerán la información con la precisión de una katana, mientras que **Optional** evitará que los inevitables valores inexistentes hagan explotar tu aplicación.

Cada archivo será una nueva misión. Algunos contendrán información perfectamente organizada; otros parecerán haber sobrevivido a una pelea entre Hulk y Juggernaut. Habrá líneas mal escritas, datos repetidos, información incompleta y formatos imposibles de entender. Tu trabajo será procesarlos con inteligencia hasta convertirlos en registros limpios, consistentes y listos para ser utilizados.

Porque una cosa está clara: si Deadpool entrega un informe lleno de errores, todos pensarán que fue culpa de Deadpool. Si tú entregas un programa lleno de errores... Deadpool dirá que también fue culpa tuya.

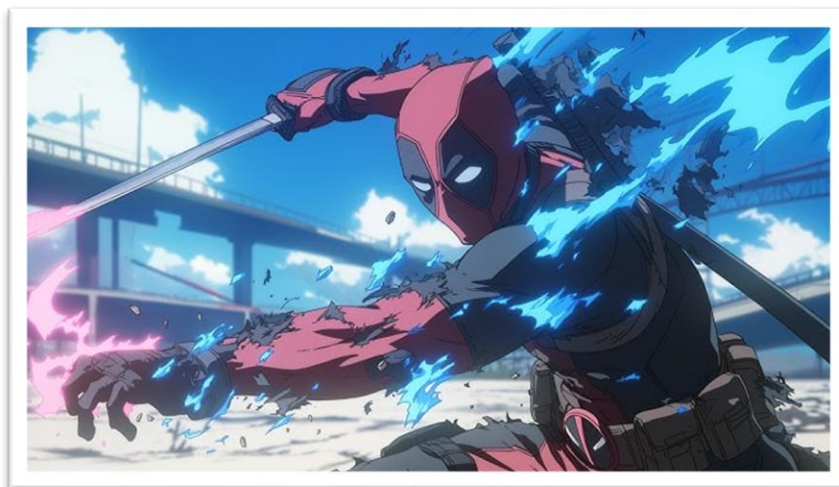
Así que afila el teclado, prepara el entorno de desarrollo y mantén cerca una reserva de chimichangas. La siguiente misión está a punto de comenzar.

III. Introducción

En este módulo aprenderás a trabajar con archivos utilizando las bibliotecas más actuales de Java. Descubrirás cómo acceder a archivos de texto, leer su contenido de forma eficiente, escribir información de manera segura y procesar grandes volúmenes de datos sin recurrir a estructuras repetitivas tradicionales. Para ello, abandonarás progresivamente el enfoque imperativo basado en bucles y adoptarás el paradigma funcional que Java incorpora desde la versión 8. Los **Streams** representan una forma diferente de pensar. En lugar de indicar paso a paso cómo recorrer una colección o un archivo, definirás una secuencia de operaciones que transformarán los datos automáticamente. Podrás filtrar registros, ordenar resultados, eliminar duplicados, realizar búsquedas, agrupar información o calcular estadísticas utilizando una sintaxis más limpia y cercana al problema que intentas resolver. Junto con ellos conocerás **Optional**, una herramienta diseñada para representar valores que pueden o no existir. Gracias a esta clase evitarás comprobaciones repetitivas contra null, reducirás la aparición de errores como NullPointerException y escribirás código mucho más seguro y fácil de comprender.

Durante las diferentes actividades del módulo trabajarás con archivos que simularán registros de misiones, bases de datos de mercenarios, inventarios de armamento, historiales de contratos y cualquier otro documento que pueda aparecer en el universo de Deadpool. Cada ejercicio planteará un nuevo reto que requerirá combinar lectura de archivos, transformación de datos y programación funcional para obtener la información solicitada.

Al finalizar este módulo serás capaz de desarrollar programas que procesen archivos de manera eficiente, segura y elegante, empleando las herramientas que utilizan actualmente muchos proyectos profesionales. Y, si todo sale bien, Deadpool seguirá llevándose el mérito... mientras tú disfrutas de un código tan limpio que incluso Coloso tendría que admitir que merece un aplauso.



A. Ejercicio 0: El Guion de la Misión

- ✓ **Directorio:** ex0/
- ✓ **Archivo:** MissionScript.java, mision.txt
- ✓ **Funciones Autorizadas:** java.nio.file.Files, java.nio.file.Path, Stream<String>, System.out.println, System.getProperty(Ruta absoluta).
- ✓ **Método:** public static void main(String[] args).
- ✓ **Restricción:**
 - ✓ Obligatorio usar Files.lines(Path path) para obtener un Stream

Objetivo - Leer un archivo con el plan de la misión y mostrarlo en consola.

mision.txt

```
=== PLAN DE LA MISIÓN ===  
1. Encender motores  
2. Verificar combustible  
3. Despegar  
4. Entrar en órbita  
5. Iniciar exploración
```

Salida Esperada

```
Directorio Actual: C:\Users\XXXX\Documents\java_modules\src\ex0  
  
=== PLAN DE LA MISIÓN ===  
1. Encender motores  
2. Verificar combustible  
3. Despegar  
4. Entrar en órbita  
5. Iniciar exploración
```

B. Ejercicio 1: Limpiando la Escena del Crimen

- ✓ **Directorio:** ex1/
- ✓ **Archivo:** SceneCleaner.java, escena.txt
- ✓ **Método:** public static void main(String[] args).
- ✓ **Funciones Autorizadas:** Stream.filter(), Stream.map(), Files.lines(), try|except, Path.of.
- ✓ **Restricción:**
 - ✓ *No uses if ni for. Todo debe ser una cadena de operaciones de Stream.*

Objetivo - Eliminar líneas con "NOISE" o "CIVILIAN" y convertir el resto a mayúsculas

escena.txt

```
Se encontraron huellas en la ventana
NOISE: interferencia de radio detectada
El sospechoso llevaba una chaqueta negra
CIVILIAN: testigo anónimo reportó el incidente
Casquillo de bala hallado cerca de la puerta
```

Salida Esperada

```
SE ENCONTRARON HUELLAS EN LA VENTANA
EL SOSPECHOSO LLEVABA UNA CHAQUETA NEGRA
CASQUILLO DE BALA HALLADO CERCA DE LA PUERTA
```

C. Ejercicio 2: El Buscador de Mercenarios (Optional)

- ✓ **Directorio:** ex2/
- ✓ **Archivo:** MercenaryFinder.java
- ✓ **Método:** String buscarMercenario(List<String> mercenarios, String nombre),
public static void main(String[] args).
- ✓ **Funciones Autorizadas:** Stream.findFirst(), filter().
- ✓ **Restricción:** Prohibido usar null. El manejo de la ausencia de resultados debe realizarse exclusivamente con orElse().

Objetivo - Buscar un nombre dentro de una lista de mercenarios. Si el nombre existe, devolverlo; si no existe, devolver un mensaje por defecto.

Datos de entrada (deben especificarse)

- Una lista de nombres de mercenarios (List<String>).
- El nombre del mercenario que se desea buscar.
- Al ser static, tenes que crear un objeto(finder).

Lista = "Geralt", "Letho", "Eskel", "Lambert", "Coën" y se busca "Letho"

Datos de salida

- Si el mercenario existe, devolver su nombre del primer encontrado.
- Si no existe, devolver el mensaje por defecto (especificar cuál, por ejemplo: "Mercenario no encontrado").

Salida Esperada

```
Letho
Mercenario no encontrado...
```


D. Ejercicio 3: El Almacén de Deadpool (I/O)

- ✓ **Directorio:** ex3/
- ✓ **Archivo:** DeadpoolVault.java
- ✓ **Método:** public static void main (String[] args)
- ✓ **Funciones Autorizadas:** try-with-resources, Files.write(), newLine(), Collectors.toList(), BufferedWriter, equals, stream.
- ✓ **Restricción:**
 - El uso de try-with-resources es obligatorio para la gestión de archivos.
 - Está prohibido dejar recursos abiertos manualmente; no se debe depender de llamadas directas a .close().
 - El filtrado de objetos debe realizarse mediante una expresión Lambda.

Objetivo - Crear un programa que gestione el almacén de objetos de Deadpool.

El programa debe:

1. Tener una lista de objetos (List<String>) que representen el loot almacenado.
2. Filtrar mediante una Lambda únicamente los objetos considerados de valor.
3. Guardar los objetos filtrados en un archivo de texto utilizando operaciones de escritura con archivos.
4. Mostrar un mensaje de confirmación indicando:
 - El nombre del archivo creado.
 - La cantidad de objetos guardados.

Lista inicial: ["Katana", "Piedra", "Oro", "Chimichanga", "Diamante"]

Filtrado: ["Katana", "Oro", "Diamante"]

Archivo generado: deadpool_loot.txt

Salida Esperada: *El archivo creado, con los datos filtrados con espacios.*

E. Ejercicio 4: Filtrado de Hechizos (Streams avanzados)

- ✓ **Directorio:** ex4/
- ✓ **Archivo:** SpellFilter.java
- ✓ **Método:** public static void main(String[] args)
- ✓ **Funciones Autorizadas:** Stream.sorted(), Stream.distinct(), Comparator
- ✓ **Restricción:**
 - Debes eliminar los hechizos duplicados utilizando distinct().
 - Debes ordenar los hechizos alfabéticamente utilizando sorted().
 - No está permitido ordenar la lista manualmente con bucles (for, while) o algoritmos propios.

Objetivo—Crear un programa que reciba una lista de hechizos desordenados y con elementos repetidos.

El programa debe:

1. Crear una lista de hechizos (List<String>).
2. Eliminar los hechizos duplicados.
3. Ordenar los hechizos alfabéticamente.
4. Mostrar por pantalla la lista final.

Data: ["Fireball", "Teleport", "Ice Blast", "Fireball", "Shield", "Teleport", "Lightning"]

Salida Esperada

```
Unique and sorted spells:
```

```
Fireball  
Ice Blast  
Lightning  
Shield  
Teleport
```

F. Ejercicio 5: Conversión de Datos (Map)

- ✓ **Directorio:** ex5/
- ✓ **Archivo:** DataConverter.java
- ✓ **Método:** public static void main(String[] args)
- ✓ **Funciones Autorizadas:** Stream.map(), Integer.parseInt, .toList()
- ✓ **Restricción:**
 - Debes utilizar una referencia a método (Integer.parseInt) para convertir los datos.
 - No está permitido realizar la conversión manualmente mediante bucles.
 - La conversión debe realizarse utilizando Stream.map().

Objetivo – Crear un programa que convierta una lista de cadenas de texto (List<String>) que representan años de servicio en una lista de números enteros (List<Integer>).

- Recorrer la lista mediante un Stream.
- Convertir cada String en un Integer usando Integer.parseInt.
- Guardar el resultado en una nueva lista.

Data: "5", "12", "3", "20", "8"

Salida Esperada

```
Converted service years:  
[5, 12, 3, 20, 8]
```

G. Ejercicio 6: Validación de Identidad (Optional + Filter)

- ✓ **Directorio:** ex6/
- ✓ **Archivo:** IdentityGuard.java
- ✓ **Método:** public static void main(String[] args)
- ✓ **Funciones Autorizadas:** Optional.filter(), Optional.isPresent(), .get().
- ✓ **Restricción**
 - No está permitido utilizar estructuras condicionales (if, else) para realizar la validación.
 - La validación debe realizarse utilizando el encadenamiento de métodos de Optional.
 - Debe utilizarse Optional.filter() para aplicar la condición de validación.

Objetivo - Crear un programa que valide si el alias de un mercenario cumple los requisitos para ser considerado un alias válido.

Un alias será aceptado si:

- Existe un valor.
 - Tiene más de 5 caracteres.
1. Crear un Optional<String>.
 2. Aplicar filter() para comprobar la longitud del alias.
 3. Comprobar el resultado mediante isPresent().
 4. Mostrar un mensaje indicando si el alias es válido o no.

Salida Esperada

```
Identity accepted: ShadowHunter  
Identity rejected: alias too short
```

H. Ejercicio 7: El Informe del Sindicato (JSON Reading)

- ✓ **Directorio:** ex07/
- ✓ **Archivo:** JsonMissionReader.java, mission.json, Mission.java
- ✓ **Funciones Autorizadas:** Jackson Library (clase ObjectMapper), Path.of, toFile(), readValue, .class, try/catch.
- ✓ **Restricción**
 - Debes utilizar ObjectMapper para leer y convertir el archivo JSON.
 - El archivo JSON debe convertirse directamente en un objeto Java de tipo Mission.
 - La clase Mission debe ser una clase POJO o un record compatible con Jackson.
 - No está permitido leer el archivo manualmente como texto y procesar los campos a mano.

Objetivo - Crear un programa que lea la información de una misión almacenada en un archivo JSON externo y la transforme en un objeto Java.

El programa debe:

1. Tener un archivo mission.json con los datos de una misión.
2. Leer el archivo utilizando Path.
3. Utilizar ObjectMapper para convertir el JSON en un objeto Mission.
4. Mostrar por consola la información de la misión cargada.

Data: mission.json

```
{
  "name": "Operation Shadow",
  "mercenary": "Deadpool",
  "difficulty": "HARD",
  "reward": 5000
}
```

Salida Esperada

```
Mission loaded successfully:
```

```
Name: Operation Shadow
Mercenary: Deadpool
Difficulty: HARD
Reward: 5000
```

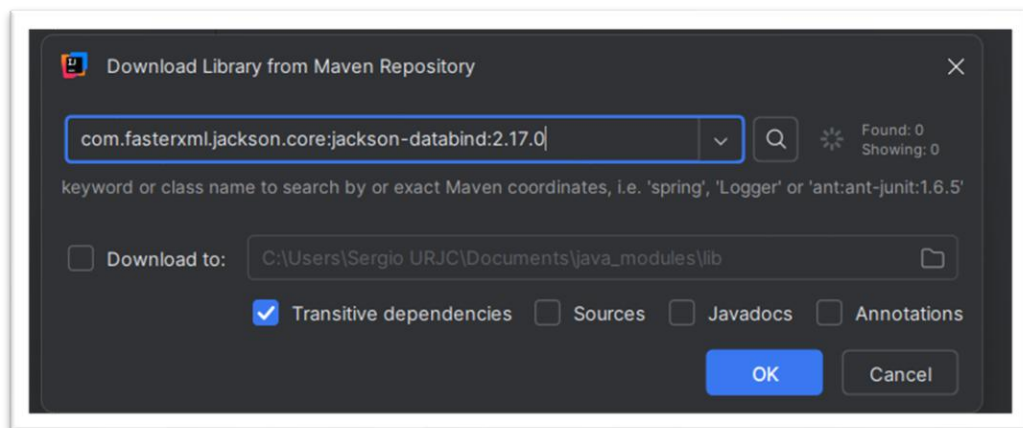
Instalación de Jackson Library

Con esto añadiremos la librería desde IntelliJ sin crear proyecto Maven.

1. En tu proyecto actual (el normal, sin Maven), ve a File > Project Structure (o **Ctrl+Alt+Shift+S**).
2. Click en Libraries (en el menú de la izquierda) → botón + → **From Maven...**
3. Te va a pedir las coordenadas Maven.

Escribe:

```
com.fasterxml.jackson.core:jackson-databind:2.17.0
```



4. Dale a buscar (la lupa) y selecciona el resultado.
5. Acepta — IntelliJ **descarga automáticamente** el .jar y sus dependencias (**jackson-core**, **jackson-annotations**) sin que tengas que ir a buscarlos a mano.
6. Aplica los cambios (Apply → OK).

Nota sobre la ejecución

```
javac -cp ".:ruta/al/jackson-databind-2.17.0.jar:ruta/al/jackson-core-2.17.0.jar:ruta/al/jackson-annotations-2.17.0.jar" JsonMissionReader.java

java -cp ".:ruta/al/jackson-databind-2.17.0.jar:ruta/al/jackson-core-2.17.0.jar:ruta/al/jackson-annotations-2.17.0.jar" JsonMissionReader
```

Pero para desarrollar y probar dentro de IntelliJ (botón "Run"), con el paso 1-6 de arriba ya te sirve — el botón verde de Run usa automáticamente las librerías que añadiste ahí, sin que tengas que tocar la terminal para nada.

G. Ejercicio 8: Exportación de Loot (JSON Writing)

- ✓ **Directorio:** ex8/
- ✓ **Archivo:** LootExporter.java, Item.java
- ✓ **Funciones Autorizadas:** Jackson (ObjectMapper),
ObjectMapper.writeValue(...), writerWithDefaultPrettyPrinter(), Path, toFile(),
try/catch.
- ✓ **Restricción:**
 - Debes utilizar la librería Jackson para generar el archivo JSON.
 - No está permitido construir el JSON manualmente mediante concatenación de cadenas.
 - La información debe escribirse en un archivo llamado loot.json.
 - El archivo debe generarse con formato legible (Pretty Print).

Objetivo

Crear un programa que convierta una lista de objetos Item almacenada en memoria en un archivo JSON.

El programa debe:

1. Crear una lista de objetos Item.
2. Utilizar ObjectMapper para convertir la lista a formato JSON.
3. Guardar el resultado en un archivo llamado loot.json.
4. Mostrar un mensaje indicando que la exportación se realizó correctamente.

Datos de entrada

```
[  
  Item("Katana", "Legendary", 5000),  
  Item("Medkit", "Common", 50),  
  Item("Grenade", "Rare", 300)  
]
```

Salida Esperada

```
Loot exported successfully: loot.json
```

A. Extra A: El Cuartel General (Collectors.groupingBy)

- ✓ **Directorio:** exA/
- ✓ **Archivo:** SquadOrganizer.java, Mercenary.java
- ✓ **Método:** Any
- ✓ **Funciones Autorizadas:** Stream, Collectors.groupingBy(), Collectors.counting(), Map<String, List<Mercenary>>, Map<String, List<Long>>, forEach
- ✓ **Restricción**
 - No está permitido utilizar bucles (for, while) para realizar el agrupamiento.
 - El agrupamiento debe realizarse exclusivamente mediante Collectors.groupingBy().
 - Como reto adicional, utiliza Collectors.counting() para obtener directamente el número de mercenarios por facción.

Objetivo

Crear un programa que reciba una lista de mercenarios y los agrupe según su facción. Cada mercenario pertenece a una única facción, por ejemplo:

- ✓ X-Force
- ✓ Mercs for Money
- ✓ Freelance

El programa debe:

1. Crear una lista de objetos Mercenary.
2. Agrupar los mercenarios por su facción utilizando Collectors.groupingBy().
3. Mostrar los mercenarios de cada facción.
4. Obtener y mostrar el número total de mercenarios por facción utilizando Collectors.counting().

Datos de entrada:

Deadpool -> Mercs for Money

Cable -> X-Force

Domino -> X-Force

Taskmaster -> Freelance

Weasel -> Mercs for Money

Salida Esperada

```
Freelance:
- Taskmaster
X-Force:
- Cable
- Domino
Mercs for Money:
- Deadpool
- Weasel

Mercenary count by faction:
Freelance -> 1
X-Force -> 2
Mercs for Money -> 2
```


B. Extra B: El Botín Total (reduce)

- ✓ **Directorio:** exB/
- ✓ **Archivo:** LootReducer.java, Item.java (*name, value*)
- ✓ **Método:** Any
- ✓ **Funciones Autorizadas:** Stream, BinaryOperator.maxBy(), reduce. get(), ifPresent()
- ✓ **Restricción**
 - No usar un bucle con un acumulador manual (int total = 0; for(...) total += x;).
 - Todas las operaciones de cálculo deben resolverse mediante reduce().
 - No utilizar métodos auxiliares como sum() o max().

Objetivo

Crear un programa que procese una lista de objetos de botín (Item) y obtenga información utilizando operaciones de reducción (reduce).

- El valor total del botín usando .reduce(0, Integer::sum)
- El objeto de mayor valor usando .reduce(BinaryOperator.maxBy(...))

Datos de entrada:

("Katana", 100)
("Medkit", 50)
("Grenade", 300)
("Gold Coin", 500)

Salida Esperada

```
Total loot value: 950  
Highest value item: Gold Coin (500)
```

C. Extra C: Sin Excusas (orElseThrow + flatMap)

- ✓ **Directorio:** exC/
- ✓ **Archivo:** NoExcusesProtocol.java, MercenaryNotFoundException.java, *Mercenary* (name, weapons).
- ✓ **Método:** public static void buscar(List<Mercenary> mercenary, String name).
- ✓ **Funciones Autorizadas:** Optional.orElseThrow(), Stream.flatMap(), excepciones custom (conectando con el módulo 02), distinct, toList, findFirst, filter, try/catch.
- ✓ **Restricción**
 - No utilizar valores por defecto con orElse().
 - No capturar la excepción con mensajes genéricos.
 - Si un mercenario no existe, debes lanzar una excepción propia:

Objetivo, parte 1 (orElseThrow):

Crear una lista de mercenarios y buscar uno por nombre.

Si existe -> Mercenary found: Deadpool

Si no existe -> Alias not found: Wolverine

Objetivo, parte 2 (flatMap):

Cada mercenario tendrá una lista de armas.

Debes transformar: *[[Katana, Pistol], [Rifle, Knife]]* en: *[Katana, Pistol, Rifle, Knife]*

No esta permitido usar for para esto.

Salida Esperada

```
Mercenary found: Deadpool
Exception:
Alias not found: Wolverine
```

```
Weapons available:
Katana
Pistol
Grenade
Rifle
Knife
Sniper Rifle
Sword
Shield
```

Data

```
[
  {
    "name": "Deadpool",
    "weapons": [
      "Katana",
      "Pistol",
      "Grenade"
    ]
  },
  {
    "name": "Cable",
    "weapons": [
      "Rifle",
      "Knife"
    ]
  },
  {
    "name": "Domino",
    "weapons": [
      "Sniper Rifle",
      "Pistol"
    ]
  },
  {
    "name": "Taskmaster",
    "weapons": [
      "Sword",
      "Shield"
    ]
  }
]
```

Ejecución

El flujo real en tu terminal

Entra en la carpeta:

Primero tienes que estar donde está el archivo. Si no, el comando `javac` te dirá "**archivo no encontrado**" y te entrarán ganas de tirar el PC por la ventana.

- ✓ `cd Desktop/proyectos_java/ex00`
- ✓ `javac FtHelloForest.java` -> La Compilación (El traductor)
- ✓ `java FtHelloForest` -> La Ejecución (El lanzamiento)